

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для XOR с ReLU на скрытых слоях: сравнение активационных функций»

Выполнил:

Студент 4 курса

Группы ИИ-26

Суховило В. С.

Проверил:

Никитин С. К.

Цель работы: На основе многослойного персептрона архитектуры 2-2-1 из ЛР №2 исследовать влияние активационной функции ReLU на скрытом слое. Провести сравнительный анализ обучения сети с Сигмоидой и ReLU по скорости сходимости, качеству обучения, форме разделяющей поверхности и устойчивости к инициализации весов.

Ход работы:

В качестве базовой конфигурации использовался многослойный персептрон 2-2-1 с функцией потерь Binary Cross-Entropy. На выходном слое в обеих конфигурациях использовалась Сигмоида. В конфигурации А на скрытом слое применялась Сигмоида, а в конфигурации В — ReLU. Обучение выполнялось методом обратного распространения ошибки в онлайн-режиме.

Вариант 3 ($c_0 = -9$, $c_1 = -8$)

A	B	$A \oplus B$
-9	-9	-9
-9	-8	-8
-8	-9	-8
-8	-8	-9

Изменения по сравнению с ЛР №2:

1. В базовой конфигурации оставлена Сигмоида на скрытом и выходном слоях.
2. В новой конфигурации на скрытом слое реализована ReLU, а на выходном слое оставлена Сигмоида.
3. Для ReLU реализована производная: градиент проходит только при положительном входе нейрона.
4. В обеих конфигурациях используется BCE и нормализованные целевые значения 0/1.
5. Выполняется серия из 5 запусков с разными seed, дополнительно контролируется наличие «мёртвых» ReLU-нейронов.

Параметры эксперимента:

Параметр	Значение
Архитектура	2-2-1
Функция потерь	Binary Cross-Entropy
Скорость обучения	0.1
Порог ошибки ϵ	0.03
Максимум эпох	20000
Seed	1, 3, 4, 5, 6
Инициализация весов	случайные малые значения

Код программы:

```
import sys
from pathlib import Path

import numpy as np
import matplotlib.pyplot as plt

# =====
# Лабораторная работа №3
# Вариант 3:  $c_0 = -9$ ,  $c_1 = -8$ 
# MLP 2-2-1 для XOR
# Сравнение Сигмоиды и ReLU на скрытом слое
# Функция потерь: Binary Cross-Entropy (BCE)
# =====

C0 = -9.0
C1 = -8.0

X_REAL = np.array([
    [-9.0, -9.0],
    [-9.0, -8.0],
    [-8.0, -9.0],
    [-8.0, -8.0]
], dtype=float)

Y_REAL = np.array([
    [-9.0],
    [-8.0],
    [-8.0],
    [-8.0],
])
```

```

        [-9.0]
    ], dtype=float)

# Параметры обучения
LEARNING_RATE = 0.1
EE = 0.03
MAX_EPOCHS = 20000
SEEDS = [1, 3, 4, 5, 6]
SHUFFLE_SEED = 42
INIT_SCALE = 0.3

# Папка для автоматически сохраняемых графиков
FIG_DIR = Path("lab3_variant3_figures")

# =====
# 1. Нормализация / денормализация
# =====

def normalize(value, c0=C0, c1=C1):
    """c0 -> 0, c1 -> 1."""
    return (value - c0) / (c1 - c0)

def denormalize(value, c0=C0, c1=C1):
    """Обратное преобразование из нормализованной шкалы."""
    return c0 + value * (c1 - c0)

X = normalize(X_REAL)
Y = normalize(Y_REAL)

# =====
# 2. Функции активации и потерь
# =====

def sigmoid(z):
    z = np.clip(z, -60.0, 60.0)
    return 1.0 / (1.0 + np.exp(-z))

def sigmoid_derivative_from_output(y):
    return y * (1.0 - y)

def relu(z):
    return np.maximum(0.0, z)

def relu_derivative(z):
    return (z > 0.0).astype(float)

def bce_loss(y_true, y_pred):
    eps = 1e-12
    y_pred = np.clip(y_pred, eps, 1.0 - eps)
    return -np.sum(
        y_true * np.log(y_pred)
        + (1.0 - y_true) * np.log(1.0 - y_pred)
    )

def classification_accuracy(y_true, y_pred):
    true_class = (y_true >= 0.5).astype(int)
    pred_class = (y_pred >= 0.5).astype(int)
    return float(np.mean(true_class == pred_class))

def mae_real(y_true_real, y_pred_norm):
    y_pred_real = denormalize(y_pred_norm)
    return float(np.mean(np.abs(y_true_real - y_pred_real)))

# =====
# 3. MLP 2-2-1
# =====

class MLP221:
    def __init__(self, hidden_activation="sigmoid", seed=1):
        if hidden_activation not in ("sigmoid", "relu"):
            raise ValueError("hidden_activation должен быть 'sigmoid' или 'relu'")

```

```

self.hidden_activation = hidden_activation
rng = np.random.RandomState(seed)

# Малые случайные веса.
self.W1 = rng.uniform(-INIT_SCALE, INIT_SCALE, size=(2, 2))

# Для ReLU небольшое положительное случайное смещение уменьшает
# вероятность мгновенного выключения обоих скрытых нейронов,
# но не исключает появления "мёртвого" нейрона в процессе обучения.
if hidden_activation == "relu":
    self.b1 = rng.uniform(0.01, INIT_SCALE, size=(1, 2))
else:
    self.b1 = rng.uniform(-INIT_SCALE, INIT_SCALE, size=(1, 2))

self.W2 = rng.uniform(-INIT_SCALE, INIT_SCALE, size=(2, 1))
self.b2 = rng.uniform(-INIT_SCALE, INIT_SCALE, size=(1, 1))

def hidden_forward(self, z1):
    if self.hidden_activation == "sigmoid":
        return sigmoid(z1)
    return relu(z1)

def hidden_derivative(self, z1, h):
    if self.hidden_activation == "sigmoid":
        return sigmoid_derivative_from_output(h)
    return relu_derivative(z1)

def forward(self, x):
    z1 = x @ self.W1 + self.b1
    h = self.hidden_forward(z1)

    z2 = h @ self.W2 + self.b2
    y_hat = sigmoid(z2)

    return z1, h, y_hat

def predict(self, x):
    return self.forward(x)[2]

def dead_neurons(self, x_train):
    """Возвращает логический массив: какие ReLU-нейроны мёртвые."""
    if self.hidden_activation != "relu":
        return np.array([False, False])

    z1 = x_train @ self.W1 + self.b1
    h = relu(z1)
    return np.all(h <= 1e-12, axis=0)

def fit(
    self,
    x_train,
    y_train,
    learning_rate=LEARNING_RATE,
    ee=EE,
    max_epochs=MAX_EPOCHS,
    shuffle_seed=SHUFFLE_SEED
):
    rng = np.random.RandomState(shuffle_seed)
    history = []
    converged = False

    for epoch in range(1, max_epochs + 1):
        indices = rng.permutation(len(x_train))

        # Онлайн-режим: обновление после каждого примера.
        for i in indices:
            x = x_train[i:i + 1]
            target = y_train[i:i + 1]

            z1, h, y_hat = self.forward(x)

            # BCE + sigmoid на выходе:
            # dE/dz2 = y_hat - target
            delta2 = y_hat - target

            # Ошибка скрытого слоя зависит от его активации.
            delta1 = (
                (delta2 @ self.W2.T)
                * self.hidden_derivative(z1, h)
            )

            self.W2 -= learning_rate * (h.T @ delta2)
            self.b2 -= learning_rate * delta2

            self.W1 -= learning_rate * (x.T @ delta1)
            self.b1 -= learning_rate * delta1

        predictions = self.predict(x_train)

```

```

        error = float(bce_loss(y_train, predictions))
        history.append(error)

        if error <= ee:
            converged = True
            break

    return {
        "epochs": epoch,
        "final_error": error,
        "history": history,
        "converged": converged
    }

# =====
# 4. Один запуск
# =====

def run_one(hidden_activation, seed):
    model = MLP221(hidden_activation=hidden_activation, seed=seed)

    training = model.fit(X, Y)
    pred_norm = model.predict(X)
    pred_real = denormalize(pred_norm)

    dead = model.dead_neurons(X)

    return {
        "activation": hidden_activation,
        "seed": seed,
        "model": model,
        "epochs": training["epochs"],
        "final_error": training["final_error"],
        "history": training["history"],
        "converged": training["converged"],
        "accuracy": classification_accuracy(Y, pred_norm),
        "mae": mae_real(Y_REAL, pred_norm),
        "pred_norm": pred_norm,
        "pred_real": pred_real,
        "dead_neurons": dead,
        "dead_count": int(np.sum(dead))
    }

# =====
# 5. Серия запусков
# =====

def run_experiments():
    sigmoid_runs = []
    relu_runs = []

    print("\n" + "=" * 84)
    print("ЛАБОРАТОРНАЯ РАБОТА №3 – ВАРИАНТ 3")
    print("Сравнение Сигмоиды и ReLU на скрытом слое")
    print("c0 = -9, c1 = -8, BCE, MLP 2-2-1")
    print("=" * 84)

    print("\nКонфигурация А: Сигмоида на скрытом слое")
    for seed in SEEDS:
        result = run_one("sigmoid", seed)
        sigmoid_runs.append(result)
        status = "ДА" if result["converged"] else "НЕТ"
        print(
            f"seed={seed:2d} | эпох={result['epochs']:5d} | "
            f"Es={result['final_error']:.6f} | "
            f"accuracy={result['accuracy']:.2f} | "
            f"MAE={result['mae']:.6f} | сходимость={status}"
        )

    print("\nКонфигурация В: ReLU на скрытом слое")
    for seed in SEEDS:
        result = run_one("relu", seed)
        relu_runs.append(result)
        status = "ДА" if result["converged"] else "НЕТ"
        dead_text = ",".join(str(i + 1) for i, d in enumerate(result["dead_neurons"]) if d)
        if not dead_text:
            dead_text = "нет"
        print(
            f"seed={seed:2d} | эпох={result['epochs']:5d} | "
            f"Es={result['final_error']:.6f} | "
            f"accuracy={result['accuracy']:.2f} | "
            f"MAE={result['mae']:.6f} | сходимость={status} | "
            f"мёртвые ReLU: {dead_text}"
        )

    return sigmoid_runs, relu_runs

```

```

# =====
# 6. Представительный запуск
# =====

def choose_representative(runs):
    successful = [r for r in runs if r["converged"]]

    if successful:
        epochs = np.array([r["epochs"] for r in successful])
        median_epochs = np.median(epochs)
        return min(successful, key=lambda r: abs(r["epochs"] - median_epochs))

    return min(runs, key=lambda r: r["final_error"])

# =====
# 7. Вывод результатов
# =====

def print_summary(sigmoid_runs, relu_runs):
    print("\n" + "=" * 84)
    print("СВОДНАЯ СТАТИСТИКА ПО 5 ЗАПУСКАМ")
    print("=" * 84)

    for title, runs in [("Сигмоида", sigmoid_runs), ("ReLU", relu_runs)]:
        successful = [r for r in runs if r["converged"]]

        print(f"\n{title}:")
        print(f"    Успешная сходимость: {len(successful)} из {len(runs)}")

        if successful:
            epochs = np.array([r["epochs"] for r in successful])
            print(f"    Минимум эпох:      {epochs.min()}")
            print(f"    Максимум эпох:      {epochs.max()}")
            print(f"    Среднее эпох:       {epochs.mean():.2f}")
            print(f"    Разброс эпох:       {epochs.max() - epochs.min()}")

            print(f"    Средняя точность:   {np.mean([r['accuracy'] for r in runs]):.4f}")
            print(f"    Средняя MAE:        {np.mean([r['mae'] for r in runs]):.6f}")

            if title == "ReLU":
                print(f"    Запусков с мёртвым нейроном: {sum(r['dead_count'] > 0 for r in runs)}")

def print_prediction_table(title, run):
    print("\n" + title)
    print("-" * 88)
    print(
        f"{'A':>7}{ 'B':>8}{ 'Цель':>10}"
        f"{'ŷ norm':>14}{ 'ŷ real':>14}{ 'Класс':>12}"
    )
    print("-" * 88)

    for x_real, target, pred_norm, pred_real in zip(
        X_REAL,
        Y_REAL.ravel(),
        run["pred_norm"].ravel(),
        run["pred_real"].ravel()
    ):
        predicted_class = C1 if pred_norm >= 0.5 else C0
        print(
            f"{x_real[0]:7.1f}{x_real[1]:8.1f}{target:10.1f}"
            f"{pred_norm:14.4f}{pred_real:14.4f}{predicted_class:12.1f}"
        )

# =====
# 8. Графики
# =====

def save_current_figure(filename):
    FIG_DIR.mkdir(exist_ok=True)
    plt.savefig(FIG_DIR / filename, dpi=180, bbox_inches="tight")

def plot_convergence(sigmoid_run, relu_run):
    plt.figure(figsize=(10, 6))

    plt.plot(
        np.arange(1, len(sigmoid_run["history"]) + 1),
        sigmoid_run["history"],
        label=f"Сигмоида, seed={sigmoid_run['seed']}"
    )

    plt.plot(
        np.arange(1, len(relu_run["history"]) + 1),
        relu_run["history"],

```

```

        label=f"ReLU, seed={relu_run['seed']}")
    )

    plt.axhline(Ee, linestyle="--", label=f"Ee = {Ee}")
    plt.yscale("log")
    plt.xlabel("Номер эпохи")
    plt.ylabel("Суммарная ошибка ВСЕ, Es")
    plt.title("Сходимость: Сигмоида и ReLU – вариант 3")
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()
    save_current_figure("01_convergence.png")

def plot_epochs(sigmoid_runs, relu_runs):
    x = np.arange(len(SEEDS))
    width = 0.36

    sigmoid_epochs = [r["epochs"] for r in sigmoid_runs]
    relu_epochs = [r["epochs"] for r in relu_runs]

    plt.figure(figsize=(10, 6))
    plt.bar(x - width / 2, sigmoid_epochs, width, label="Сигмоида")
    plt.bar(x + width / 2, relu_epochs, width, label="ReLU")

    for i, run in enumerate(sigmoid_runs):
        if not run["converged"]:
            plt.scatter(x[i] - width / 2, run["epochs"], marker="x", s=100, linewidths=2)

    for i, run in enumerate(relu_runs):
        if not run["converged"]:
            plt.scatter(x[i] + width / 2, run["epochs"], marker="x", s=100, linewidths=2)

        if run["dead_count"] > 0:
            plt.text(
                x[i] + width / 2,
                run["epochs"] * 0.96,
                "dead ReLU",
                ha="center",
                va="top",
                rotation=90,
                fontsize=8
            )

    plt.xticks(x, [str(seed) for seed in SEEDS])
    plt.xlabel("Seed")
    plt.ylabel("Количество эпох")
    plt.title("Устойчивость сходимости по 5 инициализациям – вариант 3")
    plt.grid(axis="y", alpha=0.3)
    plt.legend()
    plt.tight_layout()
    save_current_figure("02_epochs_by_seed.png")

def plot_decision_surface(run, title, filename):
    model = run["model"]

    a_values = np.linspace(-10.0, 10.0, 250)
    b_values = np.linspace(-10.0, 10.0, 250)
    aa, bb = np.meshgrid(a_values, b_values)

    grid_real = np.column_stack([aa.ravel(), bb.ravel()])
    grid_norm = normalize(grid_real)
    z = model.predict(grid_norm).reshape(aa.shape)

    plt.figure(figsize=(8, 7))
    contour = plt.contourf(
        aa,
        bb,
        z,
        levels=np.linspace(0.0, 1.0, 21),
        cmap="viridis"
    )
    plt.colorbar(contour, label="Нормализованный выход  $\hat{y}$ ")
    plt.contour(aa, bb, z, levels=[0.5], linewidths=2)

    plt.scatter(
        X_REAL[:, 0],
        X_REAL[:, 1],
        c=Y.ravel().astype(int),
        cmap="coolwarm",
        edgecolors="black",
        s=120,
        label="Обучающие примеры"
    )

    plt.xlim(-10, 10)
    plt.ylim(-10, 10)
    plt.xlabel("A")
    plt.ylabel("B")

```

```

plt.title(title)
plt.grid(True, alpha=0.2)
plt.legend()
plt.tight_layout()
save_current_figure(filename)

def plot_mae(sigmoid_run, relu_run):
    names = ["Сигмоида", "ReLU"]
    values = [sigmoid_run["mae"], relu_run["mae"]]

    plt.figure(figsize=(7, 5))
    bars = plt.bar(names, values)
    plt.ylabel("Средняя абсолютная ошибка")
    plt.title("MAE представительных запусков – вариант 3")
    plt.grid(axis="y", alpha=0.3)

    for bar, value in zip(bars, values):
        plt.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height(),
            f"{value:.5f}",
            ha="center",
            va="bottom"
        )

    plt.tight_layout()
    save_current_figure("05_mae.png")

# =====
# 9. Режим функционирования
# =====

def infer_one(model, a, b):
    x_real = np.array([[a, b]], dtype=float)
    x_norm = normalize(x_real)

    y_norm = float(model.predict(x_norm)[0, 0])
    y_real = float(denormalize(y_norm))
    nearest_class = C1 if y_norm >= 0.5 else C0

    return y_norm, y_real, nearest_class

def demonstrate_functioning(sigmoid_model, relu_model):
    print("\n" + "=" * 84)
    print("РЕЖИМ ФУНКЦИОНИРОВАНИЯ")
    print("=" * 84)

    examples = [
        (-9.0, -9.0),
        (-9.0, -8.0),
        (-8.0, -9.0),
        (-8.0, -8.0),
        (-8.5, -8.5),
        (-9.5, -8.2),
        (-7.5, -9.2)
    ]

    for a, b in examples:
        s_norm, s_real, s_class = infer_one(sigmoid_model, a, b)
        r_norm, r_real, r_class = infer_one(relu_model, a, b)

        print(f"\nA={a:.2f}, B={b:.2f}")
        print(
            f"    Сигмоида:  $\hat{y}=\{s\_norm:.4f\}$ , "
            f" $y\_real=\{s\_real:.4f\}$ , класс={s_class:.1f}"
        )
        print(
            f"    ReLU:       $\hat{y}=\{r\_norm:.4f\}$ , "
            f" $y\_real=\{r\_real:.4f\}$ , класс={r_class:.1f}"
        )

def interactive_mode(sigmoid_model, relu_model):
    print("\nВведите A и B из [-10; 10]. Пустая строка – выход.")

    while True:
        raw = input("\nA B: ").strip()
        if not raw:
            break

        try:
            a, b = map(float, raw.split())
        except ValueError:
            print("Ошибка: введите два числа через пробел.")
            continue

```

```

if not (-10 <= a <= 10 and -10 <= b <= 10):
    print("Ошибка: числа должны быть в диапазоне [-10; 10].")
    continue

s_norm, s_real, s_class = infer_one(sigmoid_model, a, b)
r_norm, r_real, r_class = infer_one(relu_model, a, b)

print(
    f"Сигмоида:  $\hat{y}=\{s\_norm:.4f\}$ , "
    f" $y\_real=\{s\_real:.4f\}$ , класс= $\{s\_class:.1f\}$ "
)
print(
    f"ReLU:  $\hat{y}=\{r\_norm:.4f\}$ , "
    f" $y\_real=\{r\_real:.4f\}$ , класс= $\{r\_class:.1f\}$ "
)

# =====
# 10. Главная программа
# =====

def main():
    sigmoid_runs, relu_runs = run_experiments()

    sigmoid_rep = choose_representative(sigmoid_runs)
    relu_rep = choose_representative(relu_runs)

    print_summary(sigmoid_runs, relu_runs)

    print_prediction_table(
        f"Представительный запуск Сигмоида (seed={sigmoid_rep['seed']}))",
        sigmoid_rep
    )
    print_prediction_table(
        f"Представительный запуск ReLU (seed={relu_rep['seed']}))",
        relu_rep
    )

    demonstrate_functioning(sigmoid_rep["model"], relu_rep["model"])

    plot_convergence(sigmoid_rep, relu_rep)
    plot_epochs(sigmoid_runs, relu_runs)
    plot_decision_surface(
        sigmoid_rep,
        f"Разделяющая поверхность: Сигмоида, seed={sigmoid_rep['seed']})",
        "03_surface_sigmoid.png"
    )
    plot_decision_surface(
        relu_rep,
        f"Разделяющая поверхность: ReLU, seed={relu_rep['seed']})",
        "04_surface_relu.png"
    )
    plot_mae(sigmoid_rep, relu_rep)

    plt.show()

    if sys.stdin.isatty():
        interactive_mode(sigmoid_rep["model"], relu_rep["model"])

if __name__ == "__main__":
    main()

```

Результат тестирования:

Активация	Seed	Эпохи	Es	Accuracy	MAE	Сходимость	Мёртвые ReLU
Сигмоида	1	5365	0.029992	1.00	0.007468	Да	—
Сигмоида	3	5896	0.029996	1.00	0.007470	Да	—
Сигмоида	4	6305	0.029991	1.00	0.007468	Да	—
Сигмоида	5	6040	0.029999	1.00	0.007470	Да	—
Сигмоида	6	5627	0.029998	1.00	0.007470	Да	—
ReLU	1	859	0.029966	1.00	0.007454	Да	нет
ReLU	3	1203	0.029861	1.00	0.007414	Да	нет
ReLU	4	1023	0.029979	1.00	0.007441	Да	нет
ReLU	5	20000	1.909576	0.75	0.333112	Нет	1
ReLU	6	800	0.029967	1.00	0.007454	Да	нет

Сводная статистика:

Показатель	Сигмоида	ReLU
Успешная сходимость	5 из 5	4 из 5
Среднее число эпох (успешные)	5846.60	971.25

Показатель	Сигмоида	ReLU
Разброс эпох (успешные)	940	403
Средняя ассигасу по 5 запускам	1.0000	0.9500
Средняя MAE по 5 запускам	0.007469	0.072575
MAE только успешных запусков	≈ 0.007469	≈ 0.007441
Запуски с мёртвым нейроном	0	1

Результаты представительных запусков:

Сигмоида, seed = 3:

A	B	Цель	$\hat{y}$ norm	$\hat{y}$ real	Класс
-9	-9	-9	0.0061	-8.9939	-9
-9	-8	-8	0.9933	-8.0067	-8
-8	-9	-8	0.9933	-8.0067	-8
-8	-8	-9	0.0103	-8.9897	-9

ReLU, seed = 1:

A	B	Цель	$\hat{y}$ norm	$\hat{y}$ real	Класс
-9	-9	-9	0.0031	-8.9969	-9
-9	-8	-8	0.9882	-8.0118	-8
-8	-9	-8	0.9882	-8.0118	-8
-8	-8	-9	0.0031	-8.9969	-9

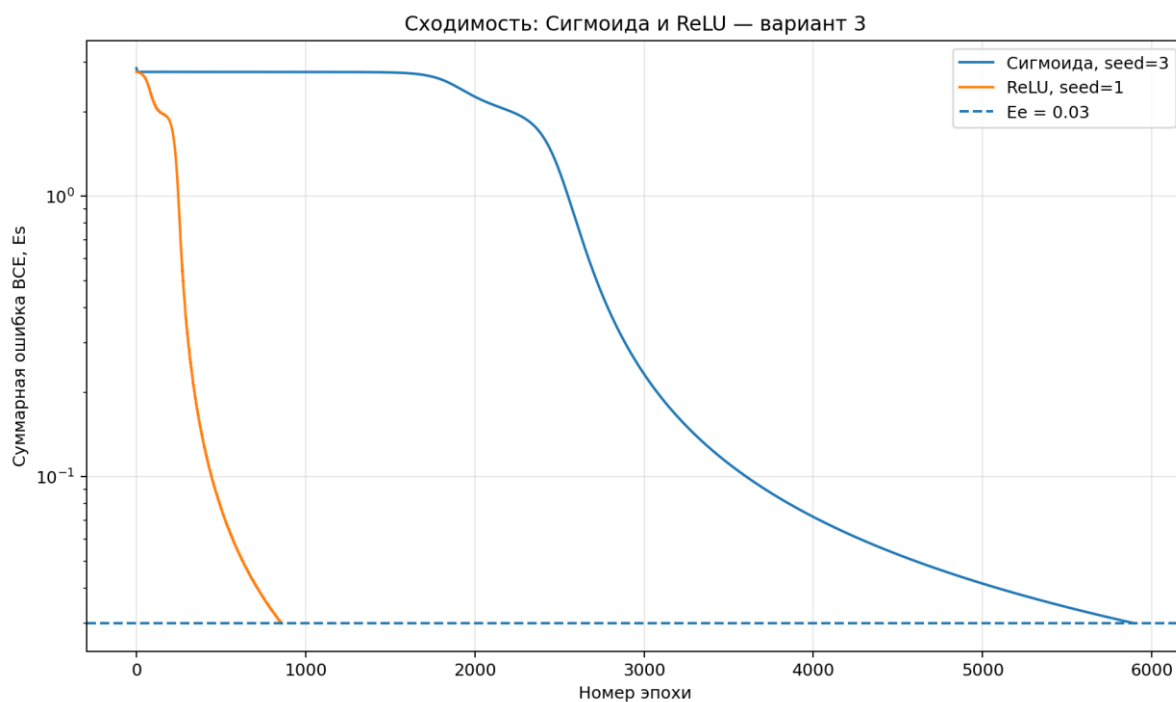


Рисунок 1 — Сравнение сходимости Сигмоиды и ReLU

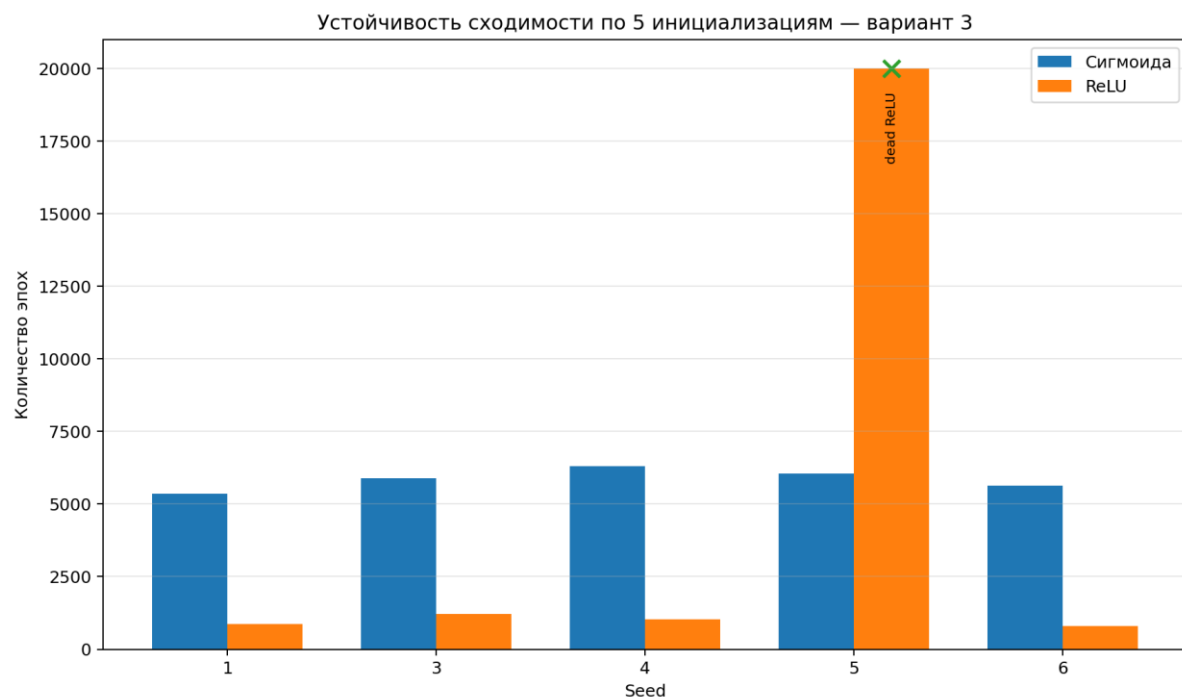


Рисунок 2 — Устойчивость сходимости по 5 случайным инициализациям

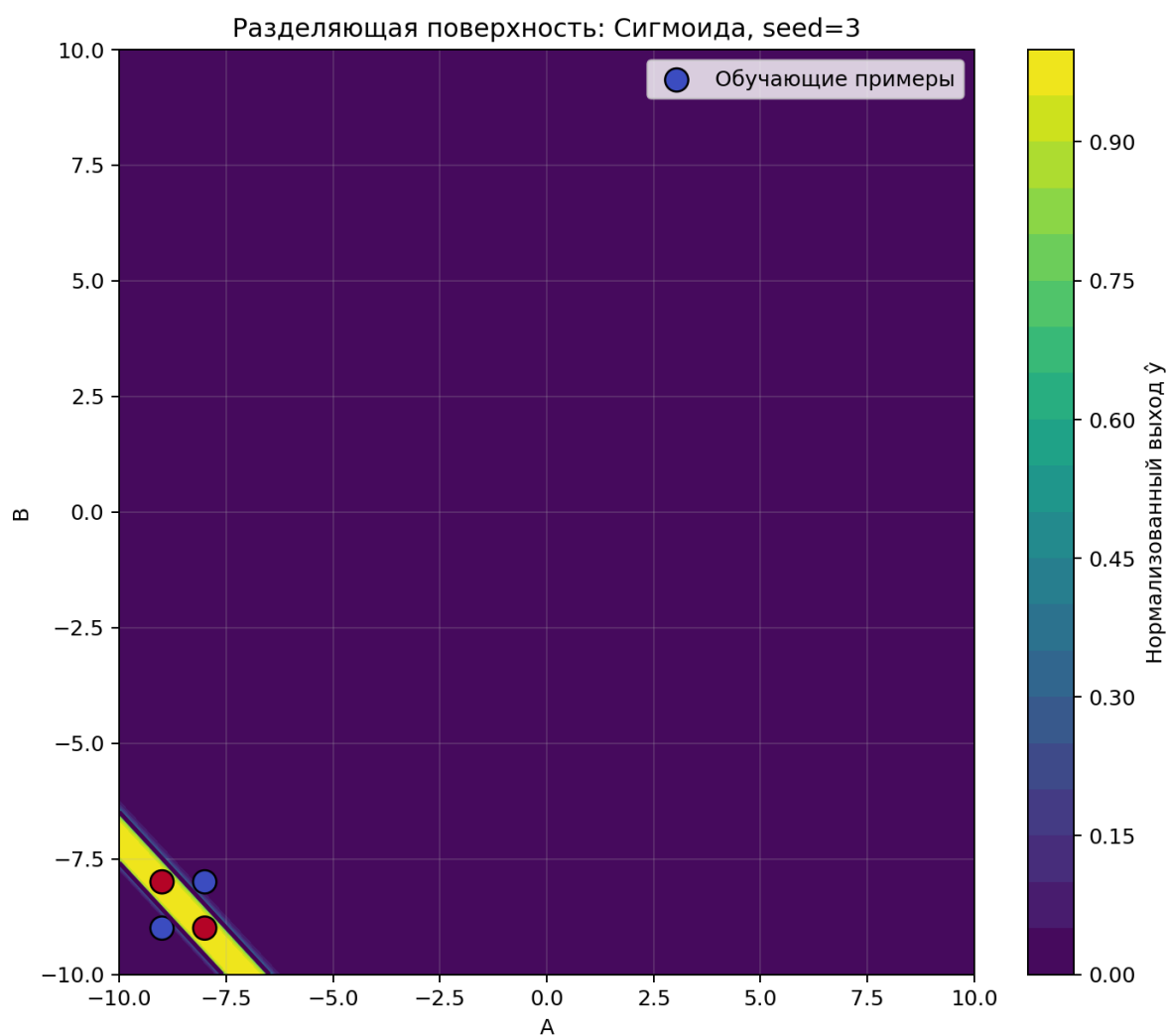


Рисунок 3 — Разделяющая поверхность для Сигмоиды

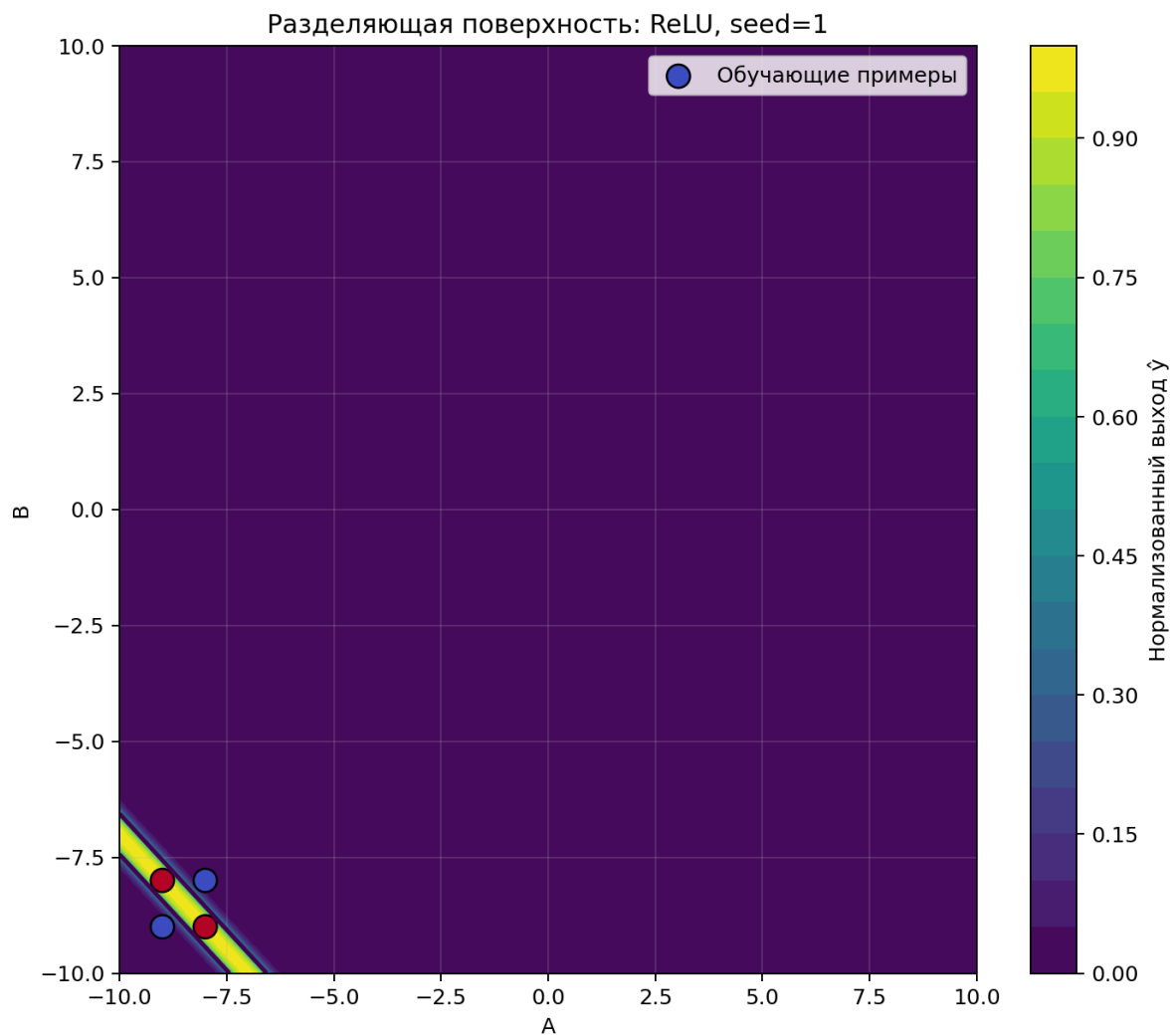


Рисунок 4 — Разделяющая поверхность для ReLU

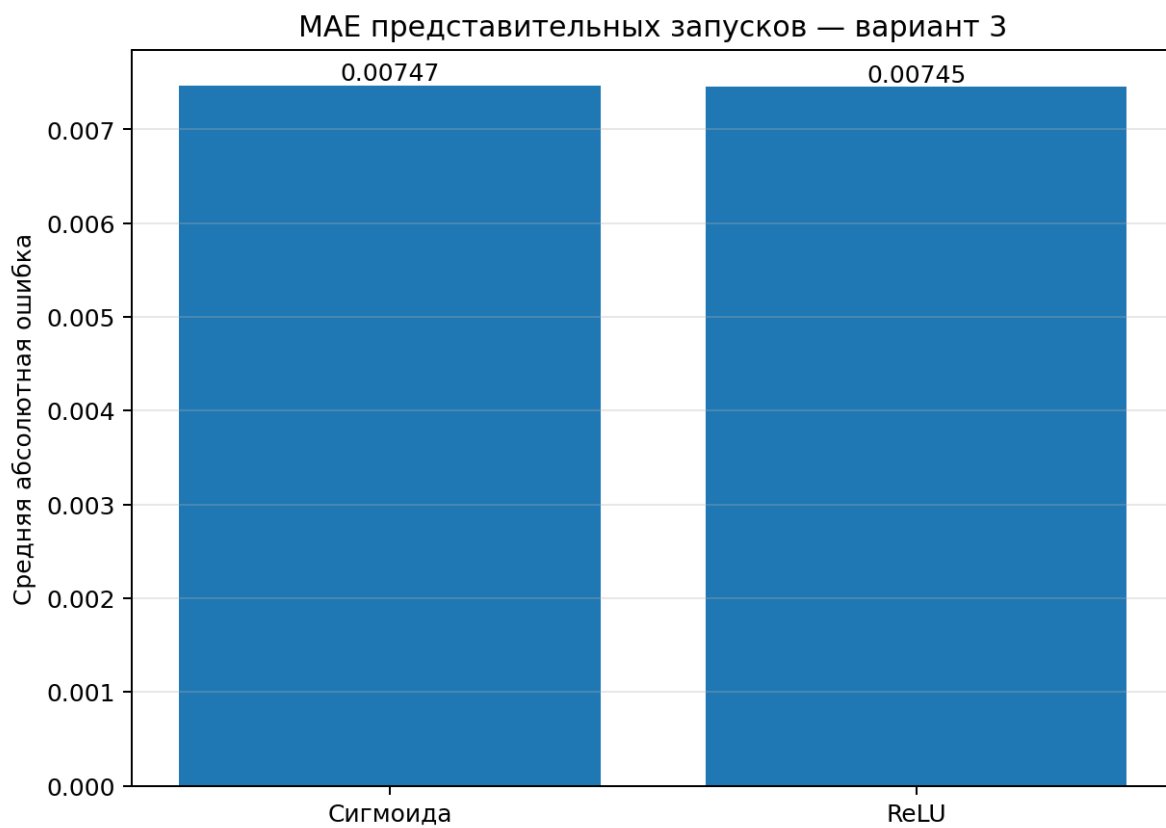


Рисунок 5 — Сравнение MAE представительных запусков

Режим функционирования сети:

A	B	Сигмоида: y_real / класс	ReLU: y_real / класс
-9.0	-9.0	-8.9939 / -9	-8.9969 / -9
-9.0	-8.0	-8.0067 / -8	-8.0118 / -8
-8.0	-9.0	-8.0067 / -8	-8.0118 / -8
-8.0	-8.0	-8.9897 / -9	-8.9969 / -9
-8.5	-8.5	-8.0067 / -8	-8.0118 / -8
-9.5	-8.2	-8.8953 / -9	-8.9365 / -9
-7.5	-9.2	-8.0506 / -8	-8.2015 / -8

Ответы на вопросы для вывода:

1. Какая активационная функция обеспечила более быструю сходимость и устойчивость?

ReLU при успешной инициализации сходилась значительно быстрее: в среднем 971.25 эпохи против 5846.60 у Сигмоиды. Однако Сигмоида оказалась устойчивее: она сошлась во всех 5 запусках, а ReLU — в 4 из 5.

2. Как отличается форма разделяющей поверхности?

У Сигмоиды переход между областями более гладкий. ReLU формирует более прямолинейные и кусочно-линейные участки границы. В данной простой сети различие заметно умеренно, поскольку на выходном слое всё равно используется Сигмоида.

3. Почему ReLU часто предпочитают Сигмоиде в глубоких сетях? Подтверждается ли это на XOR?

ReLU не насыщается на положительной области и обычно лучше передаёт градиент, поэтому обучение может идти быстрее. На XOR это подтвердилось: успешные запуски ReLU обучались примерно в несколько раз быстрее Сигмоиды.

4. Изменилась ли способность сети решать XOR при переходе на ReLU?

Нет. При удачной инициализации обе конфигурации достигли accuracy 1.00. Способность решать XOR определяется прежде всего наличием скрытого слоя, а не конкретной активационной функцией.

5. Были ли случаи «мёртвых» ReLU-нейронов?

Да. При seed = 5 один скрытый ReLU-нейрон стал постоянно выдавать 0. Из-за этого сеть не достигла порога ошибки за 20000 эпох и получила accuracy 0.75.

6. Как выбор активационной функции связан с необходимостью многослойной архитектуры?

Выбор активации влияет на скорость и устойчивость обучения, но не устраняет линейную неразделимость XOR. Для решения XOR по-прежнему необходим скрытый слой.

7. Что показывает визуализация разделяющей поверхности?

Обе успешно обученные сети формируют нелинейную область, которая отделяет две комбинации класса -8 от двух комбинаций класса -9. Это соответствует ожидаемой структуре XOR.

Вывод:

В ходе лабораторной работы был реализован многослойный персептрон 2-2-1 для XOR и проведено сравнение Сигмоиды и ReLU на скрытом слое при использовании BCE. Сигмоида продемонстрировала более высокую устойчивость к случайной инициализации и успешно сошлась во всех пяти запусках. ReLU при успешной инициализации обучалась значительно быстрее, однако в одном запуске возник «мёртвый» нейрон, из-за чего сеть не смогла полностью решить XOR. Обе успешно обученные конфигурации достигли точности 100% и сформировали корректную нелинейную разделяющую поверхность. Таким образом, ReLU может существенно ускорить обучение, но требует более внимательного отношения к инициализации, а необходимость многослойной архитектуры для XOR сохраняется независимо от выбранной активационной функции.